TP Sheet 6 - Reinforcement learning

Prepared by: Gonçalo Leão
With contributions by: Professor Armando Sousa

Exercise 1 - Wall-following using PPO

Consider the simulation scenario in the TP6.wbt Webots file, with the usual e-puck robot, equipped with a LiDAR sensor. Remember that these files were generated using the create_map.py script.

The goal is to train a robot to follow the wall on its left, similarly to ex4 from the TP2, a.k.a. a wall-following robot. However, this time, Deep Reinforcement Learning (DRL) will be used instead of a P controller to regulate the robot's angular velocity. The robot's linear velocity is a positive constant.

In this DRL scenario:

- The observation space is composed of:
  - the distance of the closest LiDAR reading on the robot's left side to an obstacle.
  - the angle relative to the robot's heading (frontal direction) of the closest LiDAR reading/ray.
  - the distance of the LiDAR ray in the front of the robot.
- The action space is the robot's angular velocity.
- The reward function…
  - rewards the robot for getting closer to the desired distance to the left wall.
  - punishes it for getting farther away from this desired distance (either by getting too close or too far to the wall).
  - punishes it for colliding with a wall.
  - punishes it for getting too far away from the left wall.

Both the action and observation spaces were normalized to the [-1; 1] range.

A custom Gymnasium environment was defined for this scenario and is already 100% done for you. Check the code carefully.

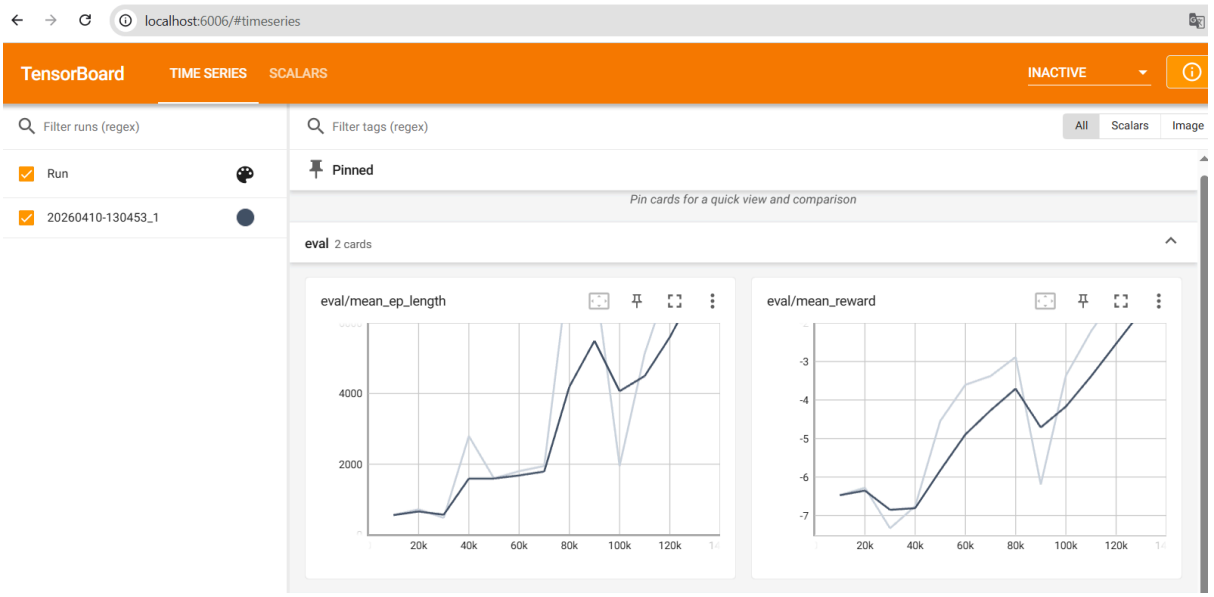
Training will be performed using the Proximal Policy Optimization (PPO) algorithm.

The training.py file is responsible for training the robot while the inference.py file is used to control to robot, once it is trained, by reading a model from a .zip file.

In this exercise, you will experiment training the robot, visualize the training results on TensorBoard and fill in the inference code in inference.py.

1. Install the required Python libraries. Be careful with compatibility between the Python version and the libraries' versions. In Python 3.8, you can install the following versions:
    - gymnasium == 1.0.0 (for the RL environment)
    - stable-baselines3 == 2.4.1 (for the RL algorithms)
    - tensorboard == 2.10.1
2. Run training.py to train the robot (the code is already 100% done) on the TP6.wbt map. Around 100k timesteps should be enough to have a good model. Notice how the robot progressively follows the wall better and better, colliding less often and moving in a straighter trajectory (less zigzagging).
3. In the tensorboard_logs directory, run the following code in the command line: python -m tensorboard.main --logdir "./"

    Then, in the Web Browser, go to the localhost URL given in the command line to visualize the training results with TensorBoard. You should get an interface similar to the one below.



4. Fill in the code for inference.py by checking the documentation for Gymnasium's Env class and Stable-Baselines3 BaseAlgorithm class.
5. Test your robot in different maps (be sure to change the path to your .zip PPO model, as we've provided a good one by default, trained with 13k timesteps for you to also test).